INDEX

Sr. No	Date	Practical	Date.	Sign
1		Write the following programs for Blockchain in Python: a. A simple client class that generates the private and public keys by using the built-in Python RSA algorithm and test it.		
		b. A transaction class to send and receive money and test it.		
		c. Create multiple transactions and display them.		
		d. Create a blockchain, a genesis block and execute it.		
		e. Create a mining function and test it.		
2		Install and configure Go Ethereum and the Mist browser.		
3		Implement and demonstrate the use of the following in Solidity: a. Variable, Operators, Loops, Decision Making, Strings, Arrays, Enums, Structs, Mappings, Conversions, Ether Units, Special Variables.		
		b. Functions, Function Modifiers, View functions, Pure Functions, Fallback Function, Function Overloading, Mathematical functions, Cryptographic functions.		

4		Implement and demonstrate the use of the following in Solidity: a. Contracts, Inheritance, Interfaces.		
		b. Libraries, Assembly, Error handling.		
5		Install hyperledger fabric and composer. Deploy and execute the application.		
6		Create your own blockchain and demonstrate its use.		

Practical No: 01

Aim: Write the following programs for Blockchain in Python

a. **A simple client class that generates the private and public keys by using the builtin Python RSA algorithm and test it.**

```
import hashlib
import random
import string
import json
import binascii
import numpy as np import pandas as pd import pylab as pl import logging import datetime import
collections
import Crypto
import Crypto.Random
from Crypto.Hash import SHA
from Crypto.PublicKey import RSA
from Crypto.Signature import PKCS1_v1_5
class Client:
def init (self):
random = Crypto.Random.new().read
self._private_key = RSA.generate(1024, random)
self._public_key = self._private_key.publickey() self._signer = PKCS1_v1_5.new(self._private_key)
@property
def identity(self): return binascii.hexlify(self._public_key.exportKey(format='DER')).decode('ascii')
class Transaction: def init ( self, sender, recipient, value ):
self.sender = sender
self.recipient = recipient self.value
= value self.time = datetime.datetime.now()
def to_dict( self ): if self.sender == "Genesis":
identity = "Genesis" else:
identity = self.sender.identity
return collections.OrderedDict( { 'sender': identity,
'recipient': self.recipient, 'value': self.value,
'time' : self.time } )
def sign_transaction( self ):
private_key = self.sender._private_key signer
= PKCS1_v1_5.new(private_key) h = SHA.new(str(self.to_dict()).encode('utf8')) return
binascii.hexlify(signer.sign(h)).decode('ascii')
Sharik = Client() Dealer = Client()
t = Transaction( Sharik, Dealer.identity, 5.0
)
signature = t.sign_transaction() print (signature)
class Block:
def init (self): self.verified_transactions = []
```

self.previous_block_hash = "" self.Nonce = ""
block0 = Block() block0.previous_block_hash
= None
Nonce = None
t0 = Transaction ("Genesis", Sharik.identity,
500.0
)
block0.verified_transactions.append (t0) digest = hash (block0) last_block_hash = digest last_block_hash = "" digest = hash (block0) last_block_hash = digest
def dump_blockchain (self):
print ("Number of blocks in the chain: " + str(len (self))) for x in range (len(TPCoins)): block_temp = TPCoins[x]
print ("block # " + str(x)) for transaction in block_temp.verified_transactions: display_transaction (transaction) def dump_blockchain (self):
print ("Number of blocks in the chain: " + str(len (self))) for x in range (len(TPCoins)): block_temp = TPCoins[x]
print ("block # " + str(x)) for transaction in block_temp.verified_transactions: display_transaction (transaction)

Output:

b. A transaction class to send and receive money and test it.

Code:

```
import hashlib
import random
import string
import json
import binascii
import numpy as np
import pandas as pd
import pylab as pl
```

```
import logging
import collections
import Crypto import Crypto.Random from Crypto.Hash import SHA from Crypto.PublicKey import
RSA from Crypto.Signature import PKCS1_v1_5
class Client: def
init (self):
random = Crypto.Random.new().read self._private_key = RSA.generate(1024, random) self._public_key
= self._private_key.publickey() self._signer = PKCS1_v1_5.new(self._private_key)
@property def identity(self):
return binascii.hexlify(self._public_key.exportKey(format='DER')).decode('ascii')
class Transaction: def init (self, sender, recipient, value): self.sender = sender self.recipient = recipient
self.value = value
self.time = datetime.datetime.now()
def to_dict(self): if self.sender == "Genesis": identity = "Genesis"
else:
identity = self.sender.identity
return collections.OrderedDict({ 'sender': identity, 'recipient': self.recipient, 'value': self.value,
'time' : self.time})
def sign_transaction(self):
private_key = self.sender._private_key signer = PKCS1_v1_5.new(private_key) h =
SHA.new(str(self.to_dict()).encode('utf8')) return binascii.hexlify(signer.sign(h)).decode('ascii')
Sharik = Client() Sharik= Client()
t = Transaction( Sharik, sharik.identity, 5.0
)
signature = t.sign_transaction()
print (signature)
```

Output:

```
>>>
================ RESTART: C:/Blockchainp/transaction_class_p1.py ===============
14442d202d67fe8dd2a1dddae345ce53cf64c258c102634c678b421ae6384b85f7886a3bc965aa4b
f4668c51334ffb1a8e5108d16c7bb1fa96d4bcdefa246f5008ea2ad9965ded9c999797e2e841aee1
493919dad5741cc61205bebfe5e9fbd9d5976b57288695ebdd36ae5e0be90730aa654a89520c7b73
a82ea7efaff01a9c
>>>
```

c. Create multiple transactions and display them.

Code:

```
import hashlib
import random
import string
import json
import binascii
import numpy as np import pandas as pd import pylab as pl import logging import datetime import
collections
import Crypto import Crypto.Random from Crypto.Hash import SHA from Crypto.PublicKey import
RSA from Crypto.Signature import PKCS1_v1_5
```

```
class Client: def
init (self):
random = Crypto.Random.new().read self._private_key = RSA.generate(1024, random) self._public_key
= self._private_key.publickey() self._signer = PKCS1_v1_5.new(self._private_key)
@property
def identity(self): return binascii.hexlify(self._public_key.exportKey(format='DER')).decode('ascii')
class Transaction: def init ( self, sender, recipient, value ):
self.sender = sender
self.recipient = recipient self.value = value self.time = datetime.datetime.now()
def to_dict( self ): if self.sender == "Genesis":
identity = "Genesis" else:
identity = self.sender.identity
return collections.OrderedDict( { 'sender': identity,
'recipient': self.recipient, 'value': self.value,
'time' : self.time } )
def sign_transaction( self ):
private_key = self.sender._private_key signer = PKCS1_v1_5.new(private_key) h =
SHA.new(str(self.to_dict()).encode('utf8')) return binascii.hexlify(signer.sign(h)).decode('ascii')
def display_transaction(transaction): #for transaction in transactions:
dict = transaction.to_dict() print ("sender: " + dict['sender'])
print (' ')
print ("recipient: " + dict['recipient']) print (' ')
print ("value: " + str(dict['value']))
print (' ')
print ("time: " + str(dict['time']))
print (' ')
transactions = [] Dinesh = Client() Ramesh = Client() Seema = Client() Vijay = Client()
t1 = Transaction( Dinesh, Ramesh.identity,
15.0 )
t1.sign_transaction() transactions.append(t1)
t2 = Transaction( Dinesh,
Seema.identity, 6.0
)
t2.sign_transaction()
t6.sign_transaction() transactions.append(t6) t7
= Transaction( Seema, Dinesh.identity, 8.0
)
t7.sign_transaction() transactions.append(t7) t8
= Transaction( Seema, Ramesh.identity, 1.0
)
t8.sign_transaction() transactions.append(t8) t9
= Transaction( Vijay, Dinesh.identity, 5.0
)
t9.sign_transaction() transactions.append(t9) t10
= Transaction( Vijay, Ramesh.identity,
3.0
)
t10.sign_transaction() transactions.append(t10)
for transaction in transactions: display_transaction (transaction) print (' ')
```

output :

```
>>>
================ RESTART: D:/blockchain/multiple transactions.py ================
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100c89181069a1a4b
c8b00f56170e3b2bd8ebd17e01a718c90ca3113b86660acd6bd160549d2b04ba7d6f4fcd4123e629
edc20f00c545bdbc8899be2265c77a34aca2224604d037c3e6c4e14c3c52bc218147ecef3360760c
8833d9614c61094e9ff5d39a66fdc56de872a9b91499cff6c85d1e79992b035c444fd4a34f724574
890203010001
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100d3f159cd4c4
758ddacad223181a0cdeae1cf85ae89affdc5c34d55e88c458e0985f7af4df52a6c6a08200c5f097
660cc08b6092269ed91390bec05ffcfca52d630d922116445fe941bc41741ea65da10ef3db23d1c0
f2a8c8c79d767a692b3d1b1a75e5a3a202e66d6054bb360d7fa113e3ab1b181009b27e7092739895
722b50203010001
-----
value: 15.0
-----
time: 2022-05-10 18:50:26.221666
-----
--------------
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100c89181069a1a4b
c8b00f56170e3b2bd8ebd17e01a718c90ca3113b86660acd6bd160549d2b04ba7d6f4fcd4123e629
edc20f00c545bdbc8899be2265c77a34aca2224604d037c3e6c4e14c3c52bc218147ecef3360760c
8833d9614c61094e9ff5d39a66fdc56de872a9b91499cff6c85d1e79992b035c444fd4a34f724574
890203010001
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100d3e92dafff8
d80c5ee2bed3c0aaf847b810663495d8ee2a630b5a90f3a29be7f269610f69297a569d65f2671d7c
8bd68219ced93780af9ea65cd65e6000a0cfb6922cb7cfb4bbe4275fbeaea85d558eec7a7b7dee86
916cba60d1483bdfc3f0e788a27787477e889a95d7a976c684c75ee101f30a505dbb34c5a05cc472
0bc7d0203010001
-----
value: 6.0
-----
time: 2022-05-10 18:50:26.223666
-----
--------------
```

```
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100d3f159cd4c4758
ddacad223181a0cdeae1cf85ae89affdc5c34d55e88c458e0985f7af4df52a6c6a08200c5f097660
cc08b6092269ed91390bec05ffcfca52d630d922116445fe941bc41741ea65da10ef3db23d1c0f2a
8c8c79d767a692b3d1b1a75e5a3a202e66d6054bb360d7fa113e3ab1b181009b27e7092739895722
b50203010001
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100a76d2fc92bb
d20d127351d92ae1e297dc3686a5f787b103849e0e5d69bac2f180d8e0c6bfe791f9f932c4c7ea05
7af0d9ebc4ca35cf45c945a0d1ac9b6af63dd2bdeaf92ece163841d9f3c1cab598a501b2d2882404
bd079b7f35bf12876e0a08fa8dfd9a0c517c6193c348468ba5d1d23c90c347efe9a77459a9618109
514410203010001
-----
value: 2.0
-----
time: 2022-05-10 18:50:26.224665
-----
--------------
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100d3e92dafff8d80
c5ee2bed3c0aaf847b810663495d8ee2a630b5a90f3a29be7f269610f69297a569d65f2671d7c8bd
68219ced93780af9ea65cd65e6000a0cfb6922cb7cfb4bbe4275fbeaea85d558eec7a7b7dee86916
cba60d1483bdfc3f0e788a27787477e889a95d7a976c684c75ee101f30a505dbb34c5a05cc4720bc
7d0203010001
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100d3f159cd4c4
758ddacad223181a0cdeae1cf85ae89affdc5c34d55e88c458e0985f7af4df52a6c6a08200c5f097
660cc08b6092269ed91390bec05ffcfca52d630d922116445fe941bc41741ea65da10ef3db23d1c0
f2a8c8c79d767a692b3d1b1a75e5a3a202e66d6054bb360d7fa113e3ab1b181009b27e7092739895
722b50203010001
-----
value: 4.0
-----
time: 2022-05-10 18:50:26.225663
-----
--------------
```

```
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100a76d2fc92bbd20
d127351d92ae1e297dc3686a5f787b103849e0e5d69bac2f180d8e0c6bfe791f9f932c4c7ea057af
0d9ebc4ca35cf45c945a0d1ac9b6af63dd2bdeaf92ece163841d9f3c1cab598a501b2d2882404bd0
79b7f35bf12876e0a08fa8dfd9a0c517c6193c348468ba5d1d23c90c347efe9a77459a9618109514
410203010001
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100d3e92dafff8
d80c5ee2bed3c0aaf847b810663495d8ee2a630b5a90f3a29be7f269610f69297a569d65f2671d7c
8bd68219ced93780af9ea65cd65e6000a0cfb6922cb7cfb4bbe4275fbeaea85d558eec7a7b7dee86
916cba60d1483bdfc3f0e788a27787477e889a95d7a976c684c75ee101f30a505dbb34c5a05cc472
0bc7d0203010001
-----
value: 7.0
-----
time: 2022-05-10 18:50:26.227657
-----
--------------
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100d3f159cd4c4758
ddacad223181a0cdeae1cf85ae89affdc5c34d55e88c458e0985f7af4df52a6c6a08200c5f097660
cc08b6092269ed91390bec05ffcfca52d630d922116445fe941bc41741ea65da10ef3db23d1c0f2a
8c8c79d767a692b3d1b1a75e5a3a202e66d6054bb360d7fa113e3ab1b181009b27e7092739895722
b50203010001
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100d3e92dafff8
d80c5ee2bed3c0aaf847b810663495d8ee2a630b5a90f3a29be7f269610f69297a569d65f2671d7c
8bd68219ced93780af9ea65cd65e6000a0cfb6922cb7cfb4bbe4275fbeaea85d558eec7a7b7dee86
916cba60d1483bdfc3f0e788a27787477e889a95d7a976c684c75ee101f30a505dbb34c5a05cc472
0bc7d0203010001
-----
value: 3.0
-----
time: 2022-05-10 18:50:26.228624
-----
--------------
```

```
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100a76d2fc92bbd20
d127351d92ae1e297dc3686a5f787b103849e0e5d69bac2f180d8e0c6bfe791f9f932c4c7ea057af
0d9ebc4ca35cf45c945a0d1ac9b6af63dd2bdeaf92ece163841d9f3c1cab598a501b2d2882404bd0
79b7f35bf12876e0a08fa8dfd9a0c517c6193c348468ba5d1d23c90c347efe9a77459a9618109514
410203010001
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100c89181069a1
a4bc8b00f56170e3b2bd8ebd17e01a718c90ca3113b86660acd6bd160549d2b04ba7d6f4fcd4123e
629edc20f00c545bdbc8899be2265c77a34aca2224604d037c3e6c4e14c3c52bc218147ecef33607
60c8833d9614c61094e9ff5d39a66fdc56de872a9b91499cff6c85d1e79992b035c444fd4a34f724
574890203010001
-----
value: 5.0
-----
time: 2022-05-10 18:50:26.232644
-----
--------------
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100a76d2fc92bbd20
d127351d92ae1e297dc3686a5f787b103849e0e5d69bac2f180d8e0c6bfe791f9f932c4c7ea057af
0d9ebc4ca35cf45c945a0d1ac9b6af63dd2bdeaf92ece163841d9f3c1cab598a501b2d2882404bd0
79b7f35bf12876e0a08fa8dfd9a0c517c6193c348468ba5d1d23c90c347efe9a77459a9618109514
410203010001
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100d3f159cd4c4
758ddacad223181a0cdeae1cf85ae89affdc5c34d55e88c458e0985f7af4df52a6c6a08200c5f097
660cc08b6092269ed91390bec05ffcfca52d630d922116445fe941bc41741ea65da10ef3db23d1c0
f2a8c8c79d767a692b3d1b1a75e5a3a202e66d6054bb360d7fa113e3ab1b181009b27e7092739895
722b50203010001
-----
value: 3.0
-----
time: 2022-05-10 18:50:26.233641
-----
--------------
```

```
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100d3e92dafff8d80
c5ee2bed3c0aaf847b810663495d8ee2a630b5a90f3a29be7f269610f69297a569d65f2671d7c8bd
68219ced93780af9ea65cd65e6000a0cfb6922cb7cfb4bbe4275fbeaea85d558eec7a7b7dee86916
cba60d1483bdfc3f0e788a27787477e889a95d7a976c684c75ee101f30a505dbb34c5a05cc4720bc
7d0203010001
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100c89181069a1
a4bc8b00f56170e3b2bd8ebd17e01a718c90ca3113b86660acd6bd160549d2b04ba7d6f4fcd4123e
629edc20f00c545bdbc8899be2265c77a34aca2224604d037c3e6c4e14c3c52bc218147ecef33607
60c8833d9614c61094e9ff5d39a66fdc56de872a9b91499cff6c85d1e79992b035c444fd4a34f724
574890203010001
-----
value: 8.0
-----
time: 2022-05-10 18:50:26.229652
-----
--------------
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100d3e92dafff8d80
c5ee2bed3c0aaf847b810663495d8ee2a630b5a90f3a29be7f269610f69297a569d65f2671d7c8bd
68219ced93780af9ea65cd65e6000a0cfb6922cb7cfb4bbe4275fbeaea85d558eec7a7b7dee86916
cba60d1483bdfc3f0e788a27787477e889a95d7a976c684c75ee101f30a505dbb34c5a05cc4720bc
7d0203010001
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100d3f159cd4c4
758ddacad223181a0cdeae1cf85ae89affdc5c34d55e88c458e0985f7af4df52a6c6a08200c5f097
660cc08b6092269ed91390bec05ffcfca52d630d922116445fe941bc41741ea65da10ef3db23d1c0
f2a8c8c79d767a692b3d1b1a75e5a3a202e66d6054bb360d7fa113e3ab1b181009b27e7092739895
722b50203010001
-----
value: 1.0
-----
time: 2022-05-10 18:50:26.230649
-----
--------------
```

d. Create a blockchain, a genesis block and execute it.
Code:

```
import hashlib
import random
import string
import json
import binascii
import numpy as np
import pandas as pd
import pylab as pl
import logging
import datetime
import collections
import Crypto import Crypto.Random from Crypto.Hash import SHA from Crypto.PublicKey import
RSA from Crypto.Signature import PKCS1_v1_5
class Client:
def init (self):
random = Crypto.Random.new().read
self._private_key = RSA.generate(1024, random) self._public_key = self._private_key.publickey()
self._signer = PKCS1_v1_5.new(self._private_key)
@property
def identity(self): return binascii.hexlify(self._public_key.exportKey(format='DER')).decode('ascii')
class Transaction: def init ( self, sender, recipient, value ):
self.sender = sender self.recipient = recipient self.value = value
self.time = datetime.datetime.now()
def to_dict( self ): if self.sender == "Genesis":
```

```
identity = "Genesis"
else:
identity = self.sender.identity
return collections.OrderedDict( { 'sender': identity,
'recipient': self.recipient,
'value': self.value, 'time' : self.time } )
def sign_transaction( self ):
private_key = self.sender._private_key signer = PKCS1_v1_5.new(private_key) h =
SHA.new(str(self.to_dict()).encode('utf8')) return binascii.hexlify(signer.sign(h)).decode('ascii') def
display_transaction(transaction):
#for transaction in transactions: dict = transaction.to_dict() print ("sender: " + dict['sender']) print (' ')
print ("recipient: " + dict['recipient']) print (' ')
print ("value: " + str(dict['value']))
print (' ')
print ("time: " + str(dict['time']))
print (' ')
class Block:
def init (self):
self.verified_transactions = [] self.previous_block_hash = "" self.Nonce = "" last_block_hash = ""
def dump_blockchain (self):
print ("Number of blocks in the chain: " + str(len (self))) for x in range (len(TPCoins)): block_temp =
TPCoins[x] print ("block # " + str(x)) for transaction in block_temp.verified_transactions:
display_transaction (transaction) print (' ')
print ('=====================================')
Dinesh = Client()
t0 = Transaction ( "Genesis",
Dinesh.identity, 500.0
)
block0 = Block() block0.previous_block_hash = None
Nonce = None block0.verified_transactions.append (t0) digest = hash (block0)
last_block_hash = digest TPCoins = [] TPCoins.append (block0) dump_blockchain(TPCoins)
```

Output:

```
>>>
========= RESTART: D:/blockchain/addingBlockchain-addingGenesisBlock.py ========
Number of blocks in the chain: 1
block # 0
sender: Genesis
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100a567f72c685
1771a0239036e044b7bdc83819fbedc5a6479b5c180c414c4dfb036e3ed0eb472ebfbe17ef61d358
29f2280006307b23c72321b8e3de8f972e887b31080ef495b0c2e03f7ed7e62e05e826ca523386c7
863f29c37b6c3d94c621823c7771dac2370578efe2e73820934238968b1eefdd038474dfcf971505
c76d70203010001
-----
value: 500.0
-----
time: 2022-05-10 20:16:05.127628
-----
--------------
=====================================
```

e. Create a mining function and test it

Codes :

```
import hashlib
import random
import string
import json
import binascii
import numpy as np
import pandas as pd
import pylab as pl
import logging
import datetime
import collections
import Crypto import Crypto.Random from Crypto.Hash import SHA from Crypto.PublicKey import
RSA
from Crypto.Signature import PKCS1_v1_5
def sha256(message):
return hashlib.sha256(message.encode('ascii')).hexdigest()
def mine(message, difficulty=1):
assert difficulty >= 1 prefix = '1' * difficulty for i in range(1000):
digest = sha256(str(hash(message)) + str(i)) if digest.startswith(prefix): print ("after " + str(i) + "
iterations found nonce: "+ digest) return digest mine ("test message", 2)
```

output :

```
>>>
========================= RESTART: C:/Blockchainp/miningfunc.py =
=====================
after 237 iterations found nonce: 113e4e3beeccf80dc416b3e1c24
557e38d83a67230b102402eb6c10d8a52e6a5
>>>
```

Practical No: 02

Aim : Install and configure Go Ethereum and the Mist browser. Develop and test a sample application.

Installing GETH (Go Ethereum)
Step 1: Go to website https://geth.ethereum.org/downloads/ Step 2: From stable releases Geth 1.5.8 (kind = installer) Step 3: once downloaded run it then click next
Step 4: Select Geth and Development tools click next Step 5: Select location to install click next
Step 6: Once Installation is finished Click Close and its done
Installing Mist Browser
Step 1: https://github.com/ethereum/mist/releases
Step 2: Under Ethereum Wallet and Mist 0.8.9 - "The Wizard" download mist-installer-0-8- 9.exe Step 3: For installation click, I agree -> next -> install
Run Mist
Step 1: Open the Mist from the start menu
Step 2: It will start downloading Blockchain data once you open it Step 3: Once it finishes downloading it is ready to use
Run Geth
Step 1: Open CMD
Step 2: Type GETH and press enter
Step 3: After it finishes loading press ctrl+c to exit the process. Step 4: Now it's ready to use

Practical No: 03

Aim : Implement and demonstrate the use of the following in Solidity

a) Variable, Operators, Loops, Decision Making, Strings, Arrays, Enums, Structs, Mappings, Conversions, Ether Units, Special Variables.

Variable codes :

```
pragma solidity ^0.5.0; contract SolidityTest { uint storedData;
// State variable constructor() public { storedData = 10;
}
function getResult() public view returns(uint){ uint a = 1; // local variable
uint b = 2; uint result = a
+ b;
return storedData; //access the state variable
}
}
```

Output :

Operators:

Arithmetic Operators codes :

```
pragma solidity ^0.5.0; contract SolidityTest { constructor() public{
}
function getResult() public view returns(uint){ uint a = 11; uint b = 22;
uint result = a + b; //arithmetic operation return result;
} }
```

Output :

```
to                  SolidityTest.getResult() 0xd2a5bC10698FD955D1Fe6cb468a17809A08fd005
execution cost      21300 gas (Cost only applies when called by a contract)
input               0xde2...92789
decoded input       {}
decoded output      {
                        "0": "uint256: 33"
                    }
logs                []
```

Comparison Operators code :

```
pragma solidity ^0.5.0;
contract SolidityTest { uint storedData; constructor() public{ storedData = 10;
}
function getResult() public view returns(string memory){ uint a = 1; // local variable uint b = 2;
uint result = a
+ b;
return integerToString(result);
}
function integerToString(uint _i) internal pure returns (string memory _uintAsString) {
if (_i == 0) { //comparison operator return "0";
} uint j = _i;
uint len;
while (j != 0) { //comparison operator len++; j /= 10;
}
bytes memory bstr = new bytes(len); uint k = len - 1;
while (_i != 0) {
bstr[k--] = byte(uint8(48 + _i % 10));
_i /= 10;
}
return string(bstr);//access local variable
} }
```

Output :

```
execution cost        22235 gas (Cost only applies when called by a contract)

input                 0xde2...92789

decoded input         {}

decoded output        {
                          "0": "string: 3"
                      }

logs                  []
```

Logical Operators :

```
pragma solidity ^0.5.0;
contract SolidityTest { uint storedData; // State variable constructor() public{ storedData = 10;
}
function getResult() public view returns(string memory){ uint a = 1; // local variable uint b = 2;
uint result = a
+ b;
return integerToString(storedData); //access the state variable
}
function integerToString(uint _i) internal pure returns (string memory) {

if (_i == 0) { return "0";
} uint j = _i;
uint len;
while (!(j == 0)) { //logical operator len++; j /= 10;
}
bytes memory bstr = new bytes(len); uint k = len - 1;
while (_i != 0) {
bstr[k--] = byte(uint8(48 + _i % 10));
_i /= 10;
}
return string(bstr);
} }
```

Output :

Bitwise Operators :

codes :

```
pragma solidity ^0.5.0;
contract SolidityTest { uint storedData; constructor() public{ storedData = 10;
}

function getResult() public view returns(string memory){ uint a = 2; // local variable uint b =
2; uint result = a & b; // bitwise operation
return integerToString(result);
}
function integerToString(uint _i) internal pure returns (string memory) { if (_i == 0) { return "0";
} uint j = _i;
uint len;
while (j != 0) { len++; j /= 10;
}
bytes memory bstr = new bytes(len); uint k = len - 1;
while (_i != 0) {
bstr[k--] = byte(uint8(48 + _i % 10));
_i /= 10;
}
return string(bstr);//access local variable
}}
```

Output:

```
input                0xde2...92789

decoded input        {}

decoded output       {
                         "0": "string: 2"
                     }

logs                 []

>
```

Assignment Operators :

```
pragma solidity ^0.5.0;
contract SolidityTest { uint storedData; constructor() public{ storedData = 10;
}
function getResult() public view returns(string memory){ uint a = 1; uint b = 2; uint result = a +
b;
return integerToString(storedData);
}
function integerToString(uint _i) internal pure returns (string memory) { if (_i == 0) { return "0";
} uint j = _i; uint len; while (j != 0) { len++;
j /= 10; //assignment operation
}
bytes memory bstr = new bytes(len); uint k = len - 1;
while (_i != 0) {
bstr[k--] = byte(uint8(48 + _i % 10));
_i /= 10;//assignment operation
}
return string(bstr); //access local variable
}
}
```

Output:

```
input                0xde2...92789
decoded input        {}
decoded output       {
                         "0": "string: 10"
                     }
logs                 []
```

Loop

While loop codes :

```
pragma solidity ^0.5.0;
contract SolidityTest { uint storedData; constructor() public{ storedData = 10;
}
function getResult() public view returns(string memory){ uint a = 10; uint b = 2; uint result = a +
b;
return integerToString(result);
}
function integerToString(uint _i) internal pure returns (string memory) {
if (_i == 0) { return "0"; } uint j = _i;
uint len;
while (j != 0) { len++; j /= 10;
}
bytes memory bstr = new bytes(len); uint k = len - 1;
while (_i != 0) { // while loop bstr[k--] = byte(uint8(48 + _i % 10));
_i /= 10;
}
return string(bstr);
}
}
```

Output :

```
input                    0xde2...92789

decoded input            {}

decoded output           {
                             "0": "string: 12"
                         }

logs                     []
```

Do while loop code :

```
pragma solidity ^0.5.0;
contract SolidityTest { uint storedData; constructor() public{ storedData = 10;
}
function getResult() public view returns(string memory){ uint a = 10; uint b = 2; uint result = a +
b;
return integerToString(result);
}
function integerToString(uint _i) internal pure returns (string memory) {
if (_i == 0) { return "0"; } uint j = _i;
uint len;
while (j != 0) { len++; j /= 10;
}
bytes memory bstr = new bytes(len); uint k = len - 1;
        do {                // do while loop
          bstr[k--] = byte(uint8(48 + _i % 10));
          _i /= 10;
        }
        while (_i != 0);
        return string(bstr);
      }
    }
```

Output :

For loop code :

```
pragma solidity ^0.5.0; contract SolidityTest {
uint storedData;
constructor() public{
storedData = 10;
}
function getResult() public view returns(string memory){ uint a = 10;
uint b = 2;
uint result = a + b;
return integerToString(result);
function integerToString(uint _i) internal pure
returns (string memory) {
if (_i == 0) {
return "0";
}
uint j=0;
}
uint len;
for (j = _i; j != 0; j /= 10) { //for loop example
len++;
}
bytes memory bstr = new bytes(len);
uint k = len - 1;          while (_i != 0) {
```

```
bstr[k--] = byte(uint8(48 + _i % 10));
_i /= 10;
return string(bstr);//access local variable }}
```

Output:

Loop control code :

```
pragma solidity ^0.5.0;

contract SolidityTest { uint storedData; constructor() public{ storedData = 10;
}
function getResult() public view returns(string memory){ uint n = 1;          uint sum = 0;

while( n < 10){           n++;    if(n == 5){      continue; // skip n in sum when it is 5.
}
sum = sum + n;
}
return integerToString(sum);
}
function integerToString(uint _i) internal pure returns (string memory) {

if (_i == 0) { return "0"; } uint j = _i;
uint len;

while (true) { len++;   j /= 10;
if(j==0){
break; //using break statement
}
}
bytes memory bstr = new bytes(len); uint k = len - 1;

while (_i != 0) {
bstr[k--] = byte(uint8(48 + _i % 10));
_i /= 10;
}

return string(bstr); }
```

Output:

```
to                  SolidityTest.getResult() 0xf8e81D47203A594245E36C48e151709F0C19fBe8
execution cost      23397 gas (Cost only applies when called by a contract)
input               0xde2...92789
decoded input       {}
decoded output      {
                        "0": "string: 49"
                    }
logs                []
```

Decision Making :
If statement codes :

```
pragma solidity ^0.5.0;
contract SolidityTest { uint storedData; constructor() public {
storedData = 10;
}
function getResult() public view returns(string memory){ uint a = 1;
uint b = 2;
uint result = a + b;
return integerToString(result);
}
function integerToString(uint _i) internal pure
returns (string memory) {
if (_i == 0) { // if statement
return "0";
}
uint j = _i;
```

```
while (j != 0) { len++; j /= 10;
}
bytes memory bstr = new bytes(len); uint k = len - 1;
while (_i != 0) {
bstr[k--] = byte(uint8(48 + _i % 10));
_i /= 10;
}
return string(bstr);//access local variable
}
}
```

Output:

```
to                  SolidityTest.getResult() 0xD7ACd2a9FD159E69Bb102A1ca21C9a3e3A5F771B

execution cost      22235 gas (Cost only applies when called by a contract)

input               0xde2...92789

decoded input       {}

decoded output      {
                        "0": "string: 3"
                    }

logs                []
```

if...else statement codes :

```
pragma solidity ^0.5.0;
contract SolidityTest { uint storedData;
constructor() public{ storedData = 10;
}
function getResult() public view returns(string memory){ uint a = 1; uint b = 2; uint result if( a >
b) { // if else statement
result = a;
} else { result = b;
}
return integerToString(result);
}
function integerToString(uint _i) internal pure returns (string memory) { if (_i == 0) { return "0";
} uint j = _i;
uint len;
while (j != 0) { len++; j /= 10;
}
bytes memory bstr = new bytes(len); uint k = len - 1;
```

```
while (_i != 0) {
bstr[k--] = byte(uint8(48 + _i % 10));
_i /= 10;
}
return string(bstr);//access local variable
}
}
```

Output :

If else if statement codes :

```
pragma solidity ^0.5.0;
contract SolidityTest { uint storedData; // State variable constructor() public { storedData = 10;
}
function getResult() public view returns(string memory)
{ uint a = 1; uint b = 2; uint c = 3; uint result
if( a > b && a > c) { // if else statement result = a;
} else if( b > a && b > c ){ result = b;
} else { result = c;
}
return integerToString(result);
}
function integerToString(uint _i) internal pure returns (string memory) {
if (_i == 0) { return "0"; } uint j = _i;
uint len;
while (j != 0) { len++; j /= 10;
}
bytes memory bstr = new bytes(len);
uint k = len - 1; while (_i != 0) {
bstr[k--] = byte(uint8(48 + _i % 10));
_i /= 10;
}
return string(bstr);//access local variable
}
}
```

Output :

```
execution cost        22235 gas (Cost only applies when called by a contract)

input                 0xde2...92789

decoded input         {}

decoded output        {
                          "0": "string: 3"
                      }

logs                  []
```

Strings codes :

```
pragma solidity ^0.5.0;

contract SolidityTest { constructor() public{
}
function getResult() public view returns(string memory){ uint a = 1;          uint b = 2;       uint
result = a + b;

return integerToString(result);
}
function integerToString(uint _i) internal pure returns (string memory) {
if (_i == 0) { return "0"; } uint j = _i;
uint len;
while (j != 0) { len++; j /= 10;
}
bytes memory bstr = new bytes(len); uint k = len - 1;
while (_i != 0) {
bstr[k--] = byte(uint8(48 + _i % 10));
_i /= 10; } return string(bstr);
}
}
```

Output :

Enums codes :

```
pragma solidity ^0.5.0;
contract test {
enum FreshJuiceSize{ SMALL, MEDIUM, LARGE } FreshJuiceSize choice;
FreshJuiceSize constant defaultChoice = FreshJuiceSize.MEDIUM;
function setLarge() public {
choice = FreshJuiceSize.LARGE;
}
function getChoice() public view returns (FreshJuiceSize) { return choice;
}
function getDefaultChoice() public pure returns (uint) { return uint(defaultChoice);
}
}
```

Output :

Structs codes :

pragma solidity ^0.5.0; contract test {
struct Book { string title; string author; uint book_id;
}
Book book;
function setBook() public { book = Book('Learn Java', 'TP', 1);
}
function getBookId() public view returns (uint) { return book.book_id;
}
}

Output :

Mappings codes :

```
pragma solidity ^0.5.0; contract LedgerBalance {
mapping(address => uint) public balances;
function updateBalance(uint newBalance) public { balances[msg.sender] = newBalance;
} }
contract Updater {
function updateBalance() public returns (uint) { LedgerBalance ledgerBalance = new
LedgerBalance();
ledgerBalance.updateBalance(10);
return ledgerBalance.balances(address(this));
}
}
```

Output :

Special variables code :

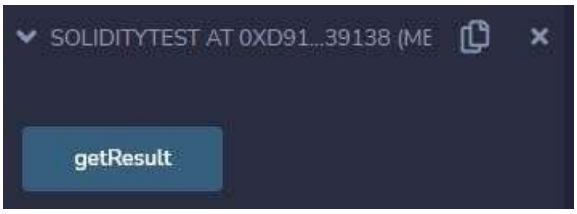
```
pragma solidity ^0.5.0; contract LedgerBalance {
mapping(address => uint) public balances;
function updateBalance(uint newBalance) public { balances[msg.sender] = newBalance;
} }
contract Updater {
function updateBalance() public returns (uint) { LedgerBalance ledgerBalance = new
LedgerBalance(); ledgerBalance.updateBalance(10);
return ledgerBalance.balances(address(this));
}
}
```

Output :

b. **Functions, Function Modifiers, View functions, Pure Functions, Fallback Function, Function Overloading, Mathematical functions, Cryptographic functions.**

Function Overloading, Mathematical functions, Cryptographic functions. Functions Code:

```
pragma solidity ^0.5.0;
contract SolidityTest { constructor() public{
}
function getResult() public view returns(string memory){ uint a = 1; uint b = 2; uint result = a
+ b;
return integerToString(result);
function integerToString(uint _i) internal pure
returns (string memory) {
if (_i == 0) {
return "0";
}         uint
j=i;
uint len;
while (j != 0)
{
len++;
j /= 10;
}
        bytes memory bstr = new bytes(len);
uint k = len - 1;
while (_i != 0) {
bstr[k--] = byte(uint8(48 + _i % 10));
_i /= 10;
}
return string(bstr);//access local variable
```



Function Modifiers:

Code:

```
pragma solidity ^0.5.0;
address owner;
string public str = "Function Modifiers Example"; constructor() public {
owner = msg.sender;
}
modifier onlyOwner { require(msg.sender == owner);
_;
}
modifier costs(uint price) { if (msg.value >= price) {
_;
}
} }
contract Register is Owner {
mapping (address => bool) registeredAddresses; uint price;
constructor(uint initialPrice) public { price = initialPrice; }
function register() public payable costs(price) { registeredAddresses[msg.sender] = true;
}
function changePrice(uint _price) public onlyOwner { price = _price;
}
}
```

Output:

Fallback Function:

Code:

```
pragma solidity ^0.5.0;
contract Test { uint public x ;
function() external { x = 1; }
}
contract Sink {
function() external payable { }
}
contract Caller {
(bool success,) = address(test).call(abi.encodeWithSignature("nonExistingFunction()"));
require(success);
// test.x is now 1
address payable testPayable = address(uint160(address(test)));
// Sending ether to Test contract,
// the transfer will fail, i.e. this returns false here. return (testPayable.send(2 ether));
}
function callSink(Sink sink) public returns (bool) { address payable sinkPayable = address(sink);
return (sinkPayable.send(2 ether));
}
string public str = "Function Callback successfully executed!";
} function callTest(Test test) public returns (bool) {
```

Output:

Call Sink:

```
[vm] from: 0x5B3...eddC4 to: Caller.callSink(address) 0xb27...07c2c value: 0 wei data: 0x07f...eddc4 logs: 0
hash: 0x841...3fc4d                                                                                    Debug

status                  true Transaction mined and execution succeed
transaction hash        0x8416d1989770aaaa925cf240531eb6e70b4442fd65093a5a05997dfdd303fc4d
from                    0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to                      Caller.callSink(address) 0xb27A31f1b0AF2946B7F582768f03239b1eC07c2c
gas                     32910 gas
transaction cost        28617 gas
execution cost          28617 gas
input                   0x07f...eddc4
decoded input           {
                            "address sink": "0x5B38Da6a701c568545dCfcB03FcB875f56beddC4"
                        }
decoded output          {
                            "0": "bool: false"
                        }
logs                    []
```

Call test:

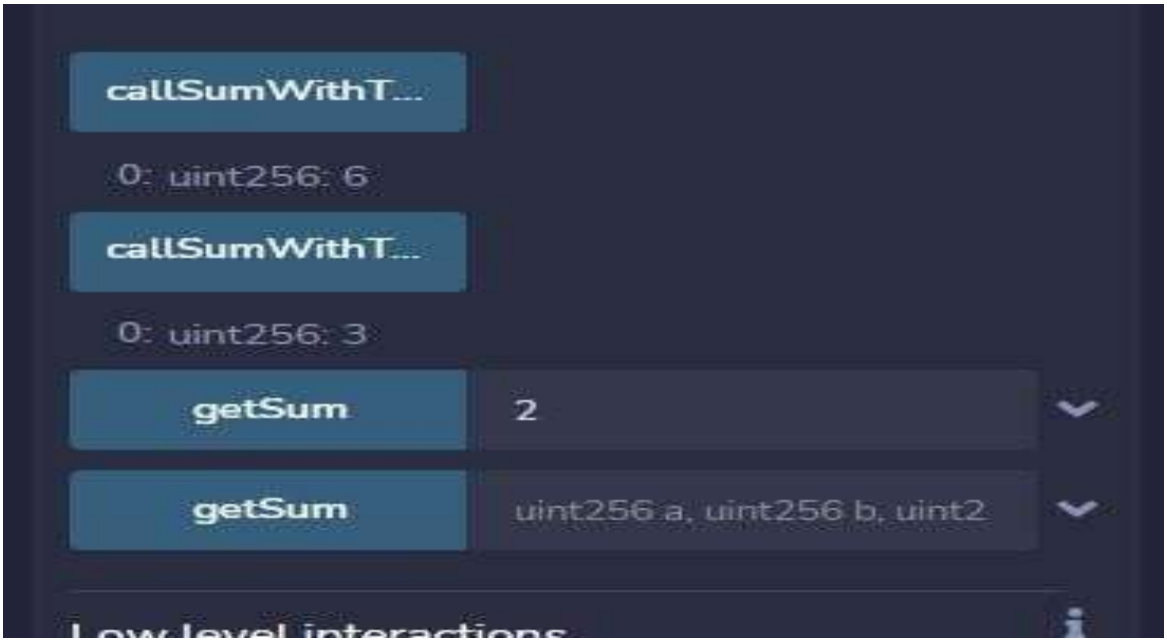
```
[vm] from: 0x5B3...eddC4 to: Caller.callTest(address) 0xb27...07c2c value: 0 wei data: 0x32e...eddc4 logs: 0
hash: 0xada...463ef                                                                                    Debug

status                  true Transaction mined and execution succeed
transaction hash        0xada7ca8f015f75468e9961e4c44d6dadc89501f34f28b9b959fd2ac8a87463ef
from                    0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to                      Caller.callTest(address) 0xb27A31f1b0AF2946B7F582768f03239b1eC07c2c
gas                     33552 gas
transaction cost        29175 gas
execution cost          29175 gas
input                   0x32e...eddc4
decoded input           {
                            "address test": "0x5B38Da6a701c568545dCfcB03FcB875f56beddC4"
                        }
decoded output          {
                            "0": "bool: false"
                        }
logs                    []
```

Function Overloading :
Codes :

```
pragma solidity ^0.5.0; contract Test {
function getSum(uint a, uint b) public pure returns(uint){
return a + b;
}
function getSum(uint a, uint b, uint c) public pure returns(uint){ return a + b + c;
}
function callSumWithTwoArguments() public pure returns(uint){ return getSum(1,2);
}
function callSumWithThreeArguments() public pure returns(uint){
return getSum(1,2,3);
}
}
```

Output :



```
input                    0x458...5ba2e
decoded input            {}
decoded output           {
                             "0": "uint256: 3"
                         }
logs                     []
```

Solidity - Cryptographic Functions

Codes :

```
pragma solidity ^0.5.0;

contract Test {
  function callKeccak256() public pure
    returns(bytes32 result){ return
    keccak256("ABC");
  }
}
```

Output :

```
input                    0x5b4...aa3ee
decoded input            {}
decoded output           {
                             "0": "bytes32: result
                         0xe1629b9dda060bb30c7908346f6af189c16773fa148d3366701fbaa35d54f3c8"
                         }
logs                     []
```

Practical No: 04

Aim: Implement and demonstrate the use of the following in Solidity:

a. Contracts, Inheritance, Interfaces. Contracts:

Code:

```
pragma solidity ^0.5.0;
contract C {
//private state variable uint private data;
//public state variable uint public info;
//constructor constructor() public { info = 10;
}
//private function
function increment(uint a) private pure returns(uint) { return a + 1; }
//public function
function updateData(uint a) public { data = a; } function getData() public view returns(uint) { return data; }
function compute(uint a, uint b) internal pure returns (uint) { return a + b; }
}
//External Contract contract D {
function readData() public returns(uint) { C c = new C();
c.updateData(7); return c.getData();
}
}
//Derived Contract contract E is C
{
uint private result; C private c;
constructor() public { c = new C();  }

function
getComputedResult()
public { result =
compute(3, 5);
  } function getResult() public view returns(uint) {
return result; } function getData() public view
returns(uint) { return c.info(); } }
```

Output:

b. Inheritance

Code:

```
pragma solidity ^0.5.0;
contract C {
//private state variable uint private data;
//public state variable uint public info;
//constructor constructor() public { info = 10;
}
//private function
function increment(uint a) private pure returns(uint) { return a + 1; }
//public function
function updateData(uint a) public { data = a; } function getData() public view returns(uint) {
return data; }
function compute(uint a, uint b) internal pure returns (uint) { return a + b; }
```

```
//Derived Contract contract E is C
{
uint private
result; C private
c; constructor()
public {
c = new C();
} function
getComputedResult()
public { result =
compute(3, 5);
   } function getResult() public view returns(uint) {
return result; } function getData() public view
returns(uint) { return c.info(); } }
```

Output: Interfaces Code:

```
pragma solidity ^0.5.0;
interface Calculator {
function getResult() external view returns(uint);
}
contract Test is Calculator {
constructor() public {} function getResult() external view returns(uint){
uint a = 1; uint b = 2; uint result = a + b; return result;
}
}
```

b**. Libraries, Assembly, Error handling.**
Libraries Code:

```
pragma solidity ^0.5.0;
library Sum {
function sumUsingInlineAssembly(uint[] memory _data) public pure returns (uint o_sum) { for
(uint i = 0; i < _data.length; ++i) {
assembly {
o_sum := add(o_sum, mload(add(add(_data, 0x20), mul(i, 0x20))))
}
}
}
}
contract Test { uint[] data;
constructor() public { data.push(1); data.push(2);
data.push(3); data.push(4); data.push(5);
}
function sum() external view returns(uint){ return Sum.sumUsingInlineAssembly(data); }
}
```

Output:

Assembly Code:

```
pragma solidity ^0.5.0; library Sum {
function sumUsingInlineAssembly(uint[] memory _data) public pure returns (uint o_sum) { for
(uint i = 0; i < _data.length; ++i) {
assembly {
o_sum := add(o_sum, mload(add(add(_data, 0x20), mul(i, 0x20))))
}
}
}
}
contract Test { uint[] data;
constructor() public { data.push(1); data.push(2);
data.push(3); data.push(4); data.push(5);
}
function sum() external view returns(uint){ return Sum.sumUsingInlineAssembly(data); }
}
```

Practical No: 05

Aim : Install hyperledger fabric and composer. Deploy and execute the application.

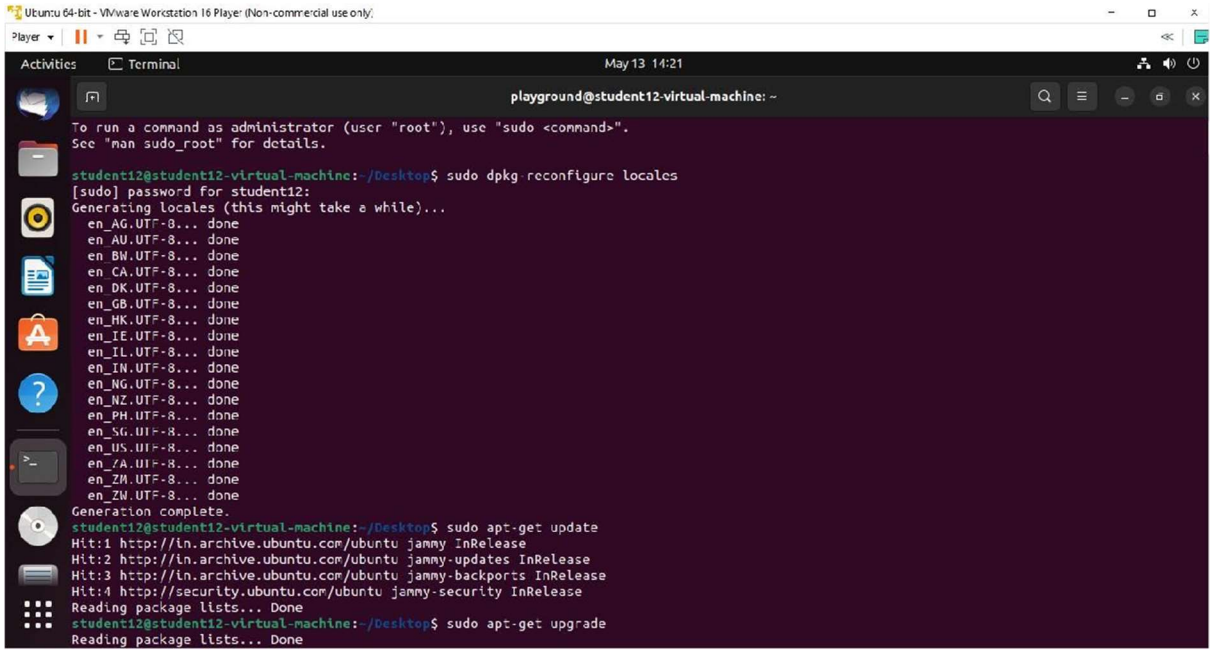
1. **Download VMware Player, you can use virtualbox as an alternate.**
2. **Download Ubuntu ISO**
3. **Install vmware player**
4. **Create VM of Ubuntu using vmware player**

Prepare the VM

$ sudo dpkg-reconfigure locales // choose en_US.UTF -8 if in doubt

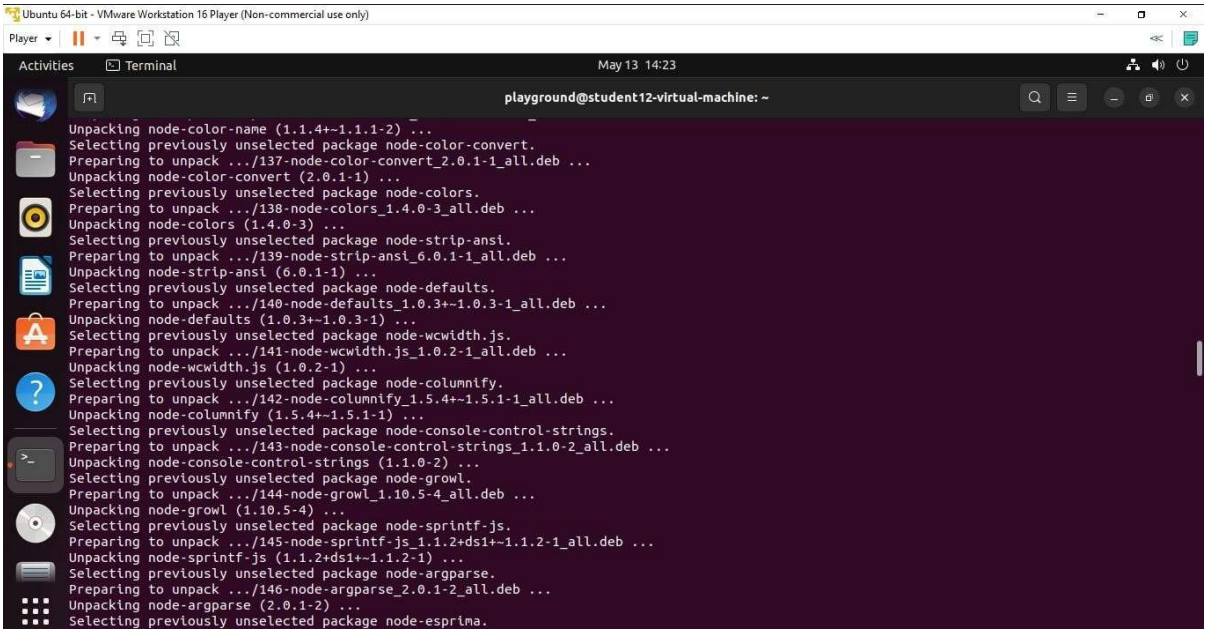
$ sudo apt-get update

$ sudo apt-get upgrade



Install pre-requists

$ sudo apt-get install curl git docker.io docker-compose golang nodejs npm



Install Docker

$ sudo usermod -a -G docker $USER

$ sudo systemctl start docker

$ sudo systemctl enable docker

$ sudo chmod 666 /var/run/docker.sock

```
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for libc-bin (2.35-0ubuntu3) ...
student12@student12-virtual-machine:~/Desktop$ sudo usermod -a -G docker $USER
student12@student12-virtual-machine:~/Desktop$ sudo systemctl start docker
student12@student12-virtual-machine:~/Desktop$ sudo systemctl enable docker
student12@student12-virtual-machine:~/Desktop$ sudo chmod 666 /var/run/docker.sock
student12@student12-virtual-machine:~/Desktop$ curl -sSL http://bit.ly/2ysbOFE | bash -s 1.4.0

Clone hyperledger/fabric-samples repo

===> Cloning hyperledger/fabric-samples repo
Cloning into 'fabric-samples'...
remote: Enumerating objects: 10217, done.
remote: Counting objects: 100% (148/148), done.
remote: Compressing objects: 100% (88/88), done.
remote: Total 10217 (delta 37), reused 119 (delta 32), pack-reused 10069
Receiving objects: 100% (10217/10217), 18.91 MiB | 2.39 MiB/s, done.
Resolving deltas: 100% (5466/5466), done.
===> Checking out v1.4.0 of hyperledger/fabric-samples

Pull Hyperledger Fabric binaries

===> Downloading version 1.4.0 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v1.4.0/hyperledger-fabric-linux-amd64-1.4.0.tar.gz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100 50.9M  100 50.9M    0     0  5615k      0  0:00:09  0:00:09 --:--:-- 6837k
==> Done.
===> Downloading version 1.5.3 platform specific fabric-ca-client binary
===> Downloading:  https://github.com/hyperledger/fabric-ca/releases/download/v1.5.3/hyperledger-fabric-ca-linux-amd64-1.5.3.tar.gz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
```

Install Hyperledger Fabric

1. Check the latest version of fabric repository, at the time of writting this post, it is 1.4 2. Install Fabric

$ curl -sSL http://bit.ly/2ysbOFE | bash -s 1.4.0

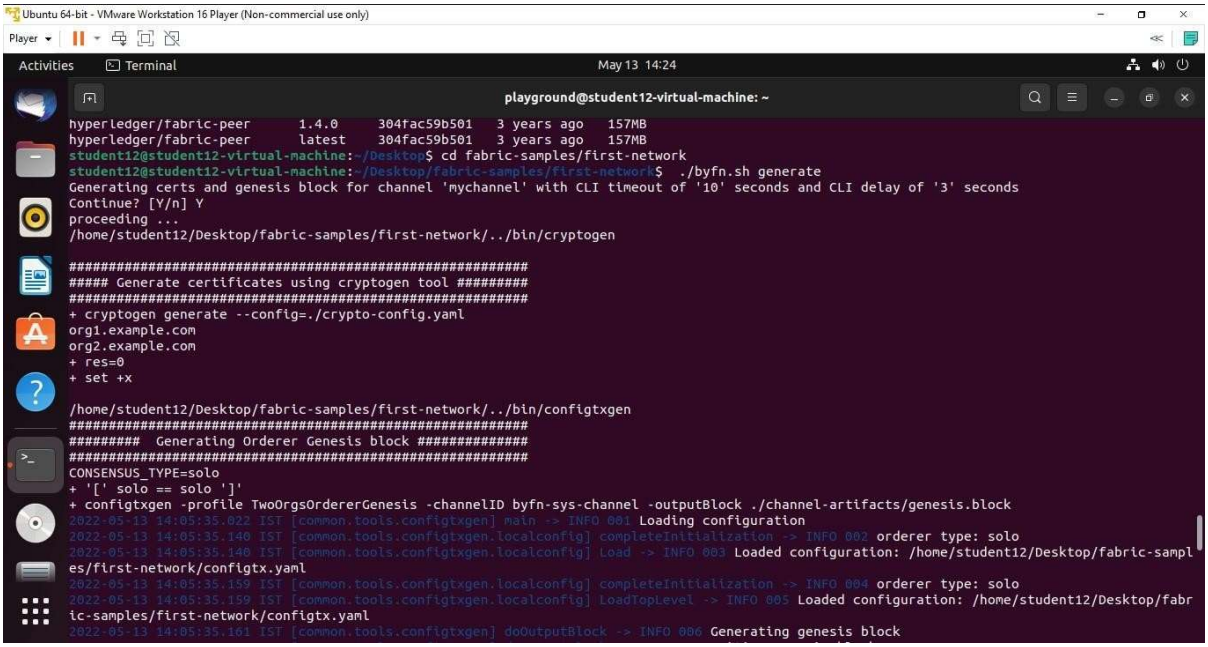
```
33f930d7053e: Already exists
7aa21e006739: Already exists
806ba27e29bb: Already exists
72b2b3c78600: Pull complete
f36a6f470154: Pull complete
19fc6d1044df: Pull complete
Digest: sha256:aee256916d0cb938d8023fa32ed2745991d32cfe79018e360f1720707ebfbdb5
Status: Downloaded newer image for hyperledger/fabric-tools:1.4.0
docker.io/hyperledger/fabric-tools:1.4.0
===> Pulling fabric ca Image
====> hyperledger/fabric-ca:1.5.3
1.5.3: Pulling from hyperledger/fabric-ca
8663204ce13b: Pull complete
fb83eec48aaa: Pull complete
59ae5613dfa3: Pull complete
Digest: sha256:f4878c8fc60e71e509fa5827f04129bbb69e519fd89bca68b1dfabfc0272d5d5
Status: Downloaded newer image for hyperledger/fabric-ca:1.5.3
docker.io/hyperledger/fabric-ca:1.5.3
===> List out hyperledger docker images
hyperledger/fabric-ca        1.5       b2aed5002b3d   5 weeks ago   68.1MB
hyperledger/fabric-ca        1.5.3     b2aed5002b3d   5 weeks ago   68.1MB
hyperledger/fabric-ca        latest    b2aed5002b3d   5 weeks ago   68.1MB
hyperledger/fabric-tools     1.4       0a44f4261a55   3 years ago   1.56GB
hyperledger/fabric-tools     1.4.0     0a44f4261a55   3 years ago   1.56GB
hyperledger/fabric-tools     latest    0a44f4261a55   3 years ago   1.56GB
hyperledger/fabric-ccenv     1.4       5b31d55f5f3a   3 years ago   1.43GB
hyperledger/fabric-ccenv     1.4.0     5b31d55f5f3a   3 years ago   1.43GB
hyperledger/fabric-ccenv     latest    5b31d55f5f3a   3 years ago   1.43GB
hyperledger/fabric-orderer   1.4       54f372205580   3 years ago   150MB
hyperledger/fabric-orderer   1.4.0     54f372205580   3 years ago   150MB
hyperledger/fabric-orderer   latest    54f372205580   3 years ago   150MB
hyperledger/fabric-peer      1.4       304fac59b501   3 years ago   157MB
hyperledger/fabric-peer      1.4.0     304fac59b501   3 years ago   157MB
```

3. Check if fabric is installed, you should see big "END" once done

$ cd fabric-samples/first-network

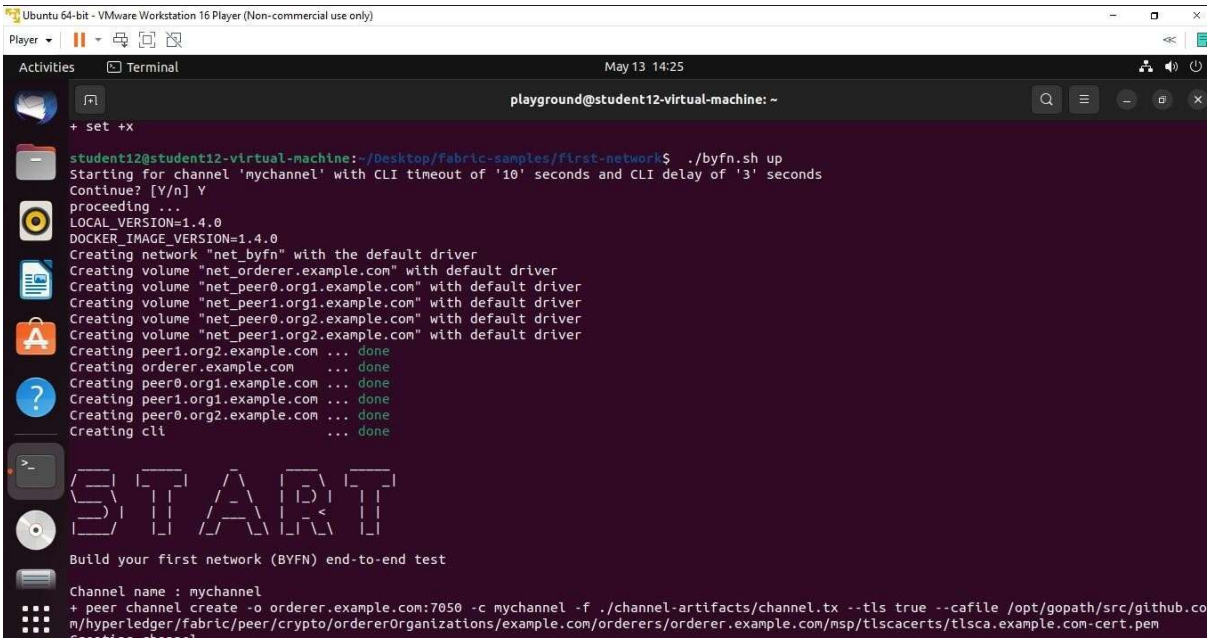
$ ./byfn.sh generate

$ ./byfn.sh up

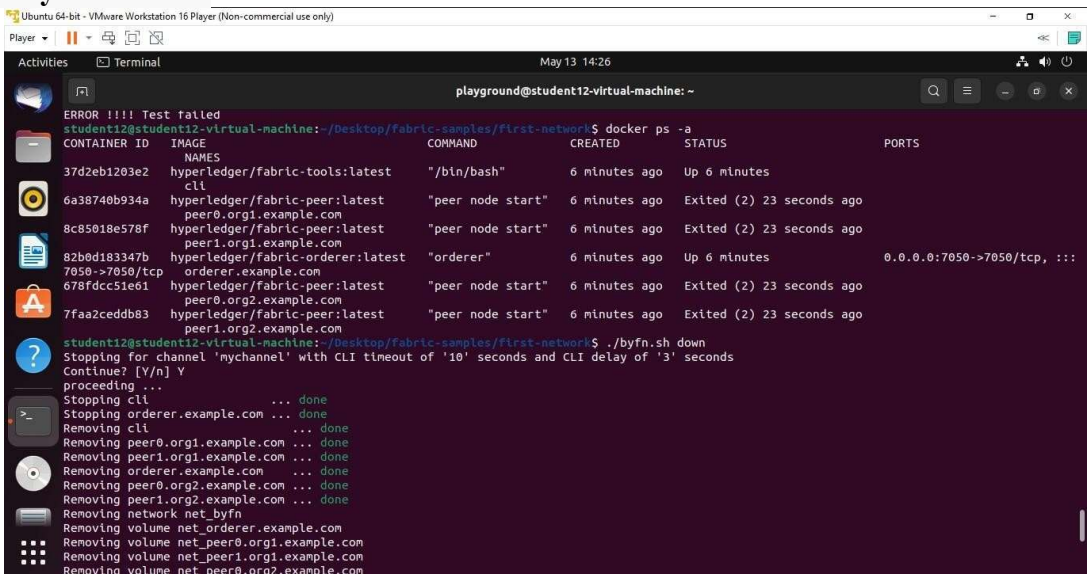


4. **Check if fabric docker is running smoothly**

5. **Stop the network**



$ ./byfn.sh down



Install Composer

1. Create new user, when asked about the full name, use something different than the full name used of the main user, to avoid confusion next time you are logging on.

   Note: if you need to keep things with the same user, jump to step number 4 followed by step number 7 directly

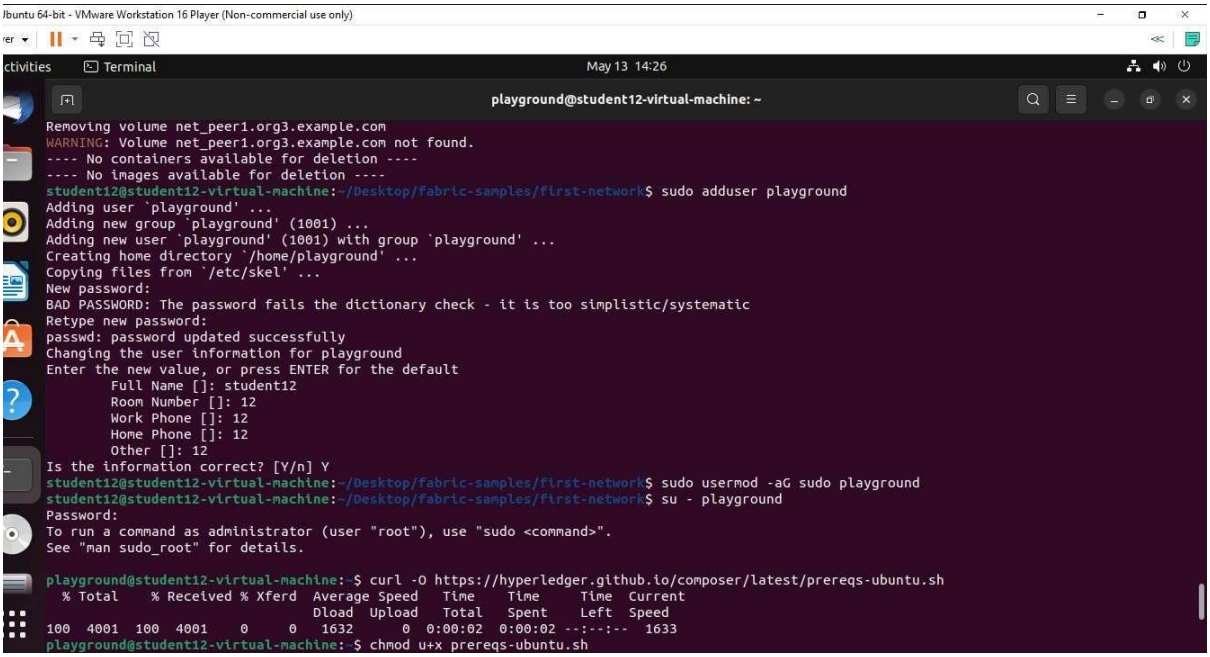
   $ sudo adduser playground

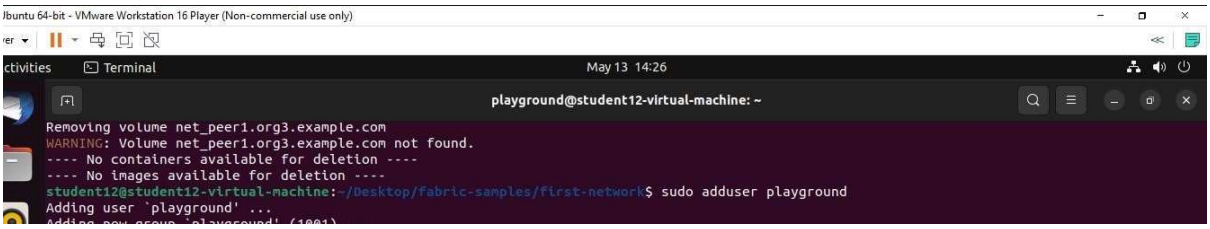
2. **Set permission for the new user**

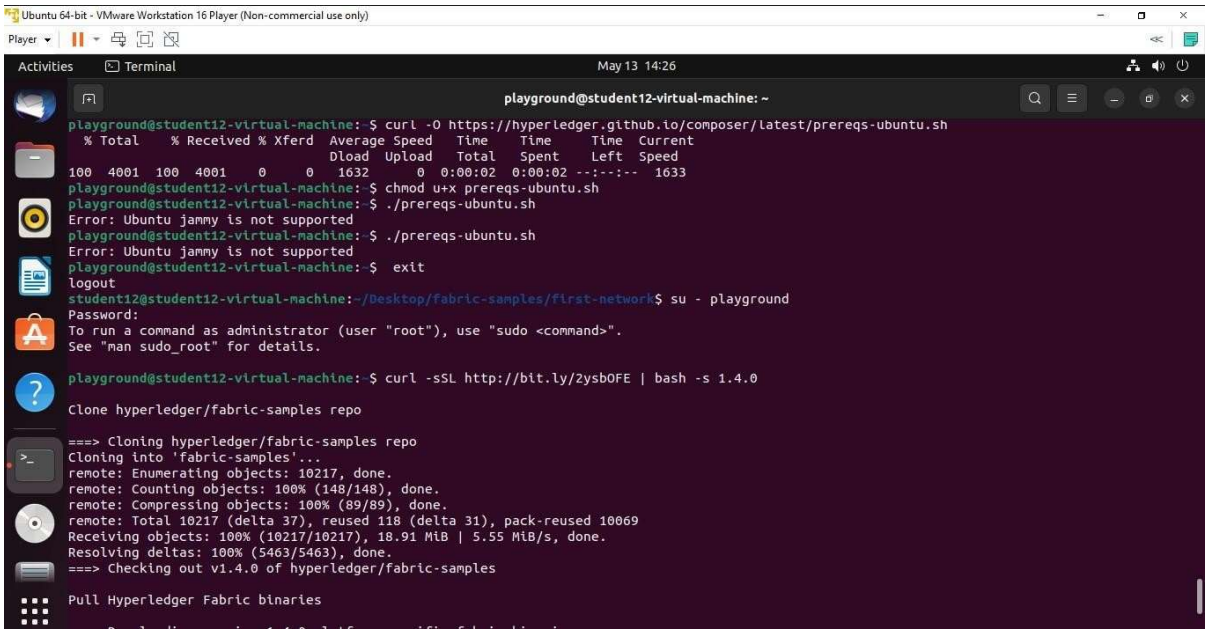
sudo usermod -aG sudo playground

3. **Login as the new user**

su - playground







4. Install the prerequisites by getting and running the script from github. It will ask for the password of "playground" account to proceed.

$ curl -O https://hyperledger.github.io/composer/latest/prereqs-ubuntu.sh

$ chmod u+x prereqs-ubuntu.sh

$ ./prereqs-ubuntu.sh

5. Logout and login with the new user to get things activated properly

$ exit

$ su - playground

6. Install components needed for running Hyperledger Fabric

$ curl -sSL http://bit.ly/2ysbOFE | bash -s 1.4.0

7. Install components needed for running Hyperledger Composer

$ npm install -g composer-cli composer-rest-server generator-hyperledger-composer yo composerplayground

8. Start Composer

$ composer-playground

Practical No: 06
Aim: Create your own blockchain and demonstrate its use.
Code :

```
class Block(object):
def init (self, index, proof_number, previous_hash, data, timestamp=None):
self.index = index self.proof_number = proof_number self.previous_hash = previous_hash
self.data = data self.timestamp = timestamp or time.time()
@property def compute_hash(self):
string_block = "{}{}{}{}{}".format(self.index, self.proof_number, self.previous_hash, self.data,
self.timestamp)
return hashlib.sha256(string_block.encode()).hexdigest() def repr (self):
return "{} - {} - {} - {} - {}".format(self.index, self.proof_number, self.previous_hash, self.data,
self.timestamp)
class BlockChain(object) def init
(self):
self.chain = [] self.current_data = []
self.nodes = set() self.build_genesis() def build_genesis(self):
self.build_block(proof_number=0, previous_hash=0) def build_block(self, proof_number,
previous_hash):
block = Block( index=len(self.chain), proof_number=proof_number,
previous_hash=previous_hash, data=self.current_data
)
self.current_data = []
self.chain.append(block) return block @staticmethod
def confirm_validity(block, previous_block): if previous_block.index + 1 != block.index:
return False
elif previous_block.compute_hash != block.previous_hash: return False elif block.timestamp <=
previous_block.timestamp:
return False return True
def get_data(self, sender, receiver, amount): self.current_data.append({
'sender': sender, 'receiver': receiver, 'amount': amount
})
return True @staticmethod def proof_of_work(last_proof):
pass @property def latest_block(self):
return self.chain[-1] def chain_validity(self):
pass def block_mining(self, details_miner):
self.get_data( sender="0", #it implies that this node has created a new block
receiver=details_miner, quantity=1, #creating a new block (or identifying the proof number) is
awared with 1
)
last_block = self.latest_block last_proof_number = last_block.proof_number proof_number =
self.proof_of_work(last_proof_number) last_hash =
last_block.compute_hash block = self.build_block(proof_number, last_hash) return vars(block)
def create_node(self, address): self.nodes.add(address) return True @staticmethod
```

```
def get_block_object(block_data): return Block( block_data['index'],
block_data['proof_number'], block_data['previous_hash'], block_data['data'],
timestamp=block_data['timestamp'
]
)
blockchain = BlockChain()
print("GET READY MINING ABOUT TO START")
print(blockchain.chain)
last_block = blockchain.latest_block last_proof_number = last_block.proof_number
proof_number = blockchain.proof_of_work(last_proof_number) blockchain.get_data(
sender="0", #this means that this node has constructed another block receiver="sharik",
amount=1, #building a new block (or figuring out the proof number) is awarded with 1
)
last_hash = last_block.compute_hash
block = blockchain.build_block(proof_number, last_hash)
print("WOW, MINING HAS BEEN SUCCESSFUL!")
print(blockchain.chain)
```

Output:

```
>>>
    ===================== RESTART: C:/Blockchainp/blockchain.py ====================
    GET READY MINING ABOUT TO START
    [0 - 0 - 0 - [] - 1652524692.432929]
    WOW, MINING HAS BEEN SUCCESSFUL!
    [0 - 0 - 0 - [] - 1652524692.432929, 1 - None - eacbafcb1ccc9faaf91424ce088a8c2b
    44497ac9da69e05c235457477fea25ab - [{'sender': '0', 'receiver': 'Dinaaz', 'amoun
    t': 1}] - 1652524692.4494965]
>>>
```